Analysis of the Implementation Process (Haoran Ma)

Program 1: Postfix Expression Evaluator

Step 1: Implementing the Stack Data Structure

The Stack class is implemented as a linked list to support postfix expression evaluation. The key features include:

1. **Node Structure**: Each node contains a value and a pointer to the next node.
2. **Core Operations**:
    - Push(): Adds a new element to the stack.
    - Pop(): Removes the top element.
    - Peek(): Retrieves the top element without removing it.
    - IsEmpty(): Checks if the stack is empty.
3. **Memory Management**: The destructor ensures that all allocated memory is properly freed to prevent memory leaks.

Step 2: Implementing the Postfix Expression Evaluator

The EvaluatePostfixExpression function processes postfix expressions using the Stack class. The steps involved are:

1. **Tokenization**: The input string is parsed into individual tokens.
2. **Processing Tokens**:
    - Numbers are converted to double and pushed onto the stack.
    - Operators pop two operands from the stack, apply the operation, and push the result back.
    - The function ensures correct operand count and detects invalid expressions.
3. **Error Handling**:
    - Invalid tokens are detected and reported.
    - Division by zero is explicitly handled.
    - The final result is retrieved and validated before returning.

Complexity Analysis for Program 1

- **Push operation (O(1))**: Each push onto the stack adds an element in constant time.
- **Pop operation (O(1))**: Removing the top element also operates in constant time.
- **Evaluation (O(n))**: Since each token is processed once, the time complexity of evaluating a postfix expression is linear with respect to the number of tokens.

Program 2: Deque Data Structure

Step 1: Implementing the Deque Data Structure

The Deque class was implemented to support basic operations such as insertion, deletion, and element access. The key considerations in its design were:

1. **Memory Management**: The deque uses a dynamically allocated array to store elements. The resize function ensures efficient memory allocation.
2. **Circular Buffer Mechanism**: The implementation uses head_ and tail_ indices to maintain a circular buffer, allowing efficient insertions and deletions from both ends.
3. **Functions Implemented**:
    - PushFront(), PushBack(), PopFront(), and PopBack() handle element insertion and deletion.
    - Front(), Back(), and operator[] allow access to elements.
    - ShrinkToFit() ensures that unused memory is freed when necessary.

Complexity Analysis for Program 2

- **PushFront and PushBack (O(1))**: Adding elements at either end is a constant time operation.
- **PopFront and PopBack (O(1))**: Removing elements at either end is also performed in constant time.
- **Access (O(1))**: The Front(), Back(), and indexed access are all constant time operations.
- **Resizing (O(n))**: When resizing occurs, copying elements takes linear time, but this happens infrequently, making the amortized cost per operation still close to O(1).

Testing and Validation for Program 2

The implementation was tested using Google Test framework. The test_deque.cc file includes unit tests for the Deque class. The key test cases include:

1. **Basic Operations**:
    - Checking if an empty deque behaves correctly.
    - Verifying insertion and retrieval of elements.
2. **Edge Cases**:
    - Testing boundary conditions like inserting at both ends.
    - Ensuring correct behavior when shrinking the deque.

Conclusion

This implementation successfully builds both **Postfix Expression Evaluator (Program 1)** and **Deque Data Structure (Program 2)**. The approach ensures efficiency and correctness through careful memory management and systematic testing. The complexity analysis demonstrates that both implementations provide optimal performance for their respective use cases.